MEV Frontrunning Analysis Report

1. Introduction

Maximum Extractable Value (MEV) refers to the additional value that can be captured by influencing transaction ordering during block construction. On automated market maker (AMM) protocols such as Uniswap V2, the outcome of a swap depends not only on pool reserves and trade size, but also on where the transaction appears within the block. Transaction ordering is therefore economically meaningful and creates room for strategies such as frontrunning, back-running, and sandwiching.

This project examines MEV-related behavior on Uniswap V2 using historical Ethereum mainnet event logs. The analysis focuses on five major trading pairs: `WETH_USDC`, `WETH_USDT`, `WETH_DAI`, `WETH_WBTC`, and `USDC_USDT`. Based on structured swap records reconstructed from on-chain event data, the study identifies four suspicious patterns:

- Sandwich attacks
- Displacement frontrunning
- Arbitrage / back-running
- Suppression

The goal is not to infer intent from isolated transactions. Instead, the report develops a reproducible workflow for studying ordering-sensitive behavior on AMM markets. By combining swap ordering, gas-price signals, and profit-related estimates, the analysis evaluates how closely the observed patterns align with known MEV strategies.

2. System Workflow

The analysis is organized as a three-stage pipeline: data loading and preprocessing, MEV pattern detection, and post-analysis.

1. **Data loading and preprocessing**
   Historical Uniswap V2 `Swap` and `Sync` event data are loaded from pair-specific CSV files. Numeric fields are cleaned, transactions are ordered by block position, and trader entities are heuristically identified.
2. **Detection of suspicious transaction-ordering patterns**
   The detector scans block-level swap sequences and applies four rule-based heuristics to identify

sandwich attacks, displacement frontrunning, arbitrage / back-running, and suppression.

3. **Post-analysis and visualization**
   After detection, the system estimates profits where possible, analyzes gas-price behavior, summarizes pair-level statistics, and generates charts and export files.

Even without mempool data, this workflow provides a practical basis for comparing suspicious transaction-ordering patterns across major Uniswap V2 pairs. That limitation still matters, but ordered on-chain logs are sufficient to support a meaningful comparative analysis.

3. Data Collection and Processing

3.1 Crawler Outcome (Etherscan API + Uniswap V2 `Pair`)

The crawler is implemented in Python (`uniswap_fetcher/`) and uses the Etherscan `getLogs` API to collect Uniswap V2 pair events from Ethereum mainnet. Rather than emphasizing API mechanics, this section summarizes the resulting dataset.

The crawler covers the period from early Uniswap V2 history to March 2026, spanning **14,639,489 blocks**. In total, **2,690,493** raw logs were fetched, and **2,690,473** decoded records remained after filtering and normalization. The five trading pairs included in the analysis are `WETH_USDC`, `WETH_USDT`, `WETH_DAI`, `WETH_WBTC`, and `USDC_USDT`.

Under key rotation, rate limiting, and retry control, the full collection process finished in about **8 hours**. To support reproducibility, the fetched data were also published on Kaggle for independent verification: **https://www.kaggle.com/datasets/chickenbilibili/uniswapv2-exchange-history**

In implementation terms, stable collection depends on adaptive block-window pagination, checkpoint resume, and per-key throttling. Decoded logs are exported into `Swap`, `Sync`, `Mint`, and `Burn` CSV files under `data/<PAIR_NAME>/`.

Pair contracts configured for this project (each row is one `UniswapV2Pair` deployed address):

`name` (output folder)	Pair contract (checksummed / config as stored)
`WETH_USDC`	`0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc`
`WETH_USDT`	`0x0d4a11d5EEaaC28EC3F61d100daF4d40471f1852`
`WETH_DAI`	`0xA478c2975Ab1Ea89e8196811F51A7B7Ade33eB11`
`WETH_WBTC`	`0xBb2b8038a1640196FbE3e38816F3e67Cba72D940`

`name` (output folder)	Pair contract (checksummed / config as stored)
`USDC_USDT`	`0x3041CbD36888bECc7bbCBc0045E3B1f144466f5f`

The default **maximize** mode starts near the Uniswap V2 factory deployment (`maximize_from_block: 10000835`) and continues toward the chain tip unless a finite `(from_block, to_block)` range is specified in `config.yaml` .

3.2 Event Types

The project uses four Uniswap V2 event types:

- **Swap**: records token exchange activity and serves as the primary input for MEV detection.
- **Sync**: records reserve updates and supports reserve tracking and ETH price reconstruction.
- **Mint**: records liquidity additions.
- **Burn**: records liquidity removals.

Among these, `Swap` and `Sync` provide the core information used in the final analysis.

3.3 Entity Identification

The effective trading entity is defined heuristically. If the `sender` address is a known Uniswap V2 router, the system uses the `to` address as the actual entity; otherwise, it uses the `sender` itself. This does not fully solve attribution, but it is a reasonable compromise for grouping router-mediated swaps under the participant most directly involved in the trade outcome.

3.4 Transaction Ordering

To reconstruct execution order inside each block, swaps are sorted by `block_number` , `tx_index` , and `log_index` . Formally, the in-block order of a transaction is represented as:

$$\mathrm{ord}(x) = (\mathrm{txIndex}_x,\ \mathrm{logIndex}_x)$$

This ordering is central to the analysis because every detection rule depends on the relative position of swaps within the same block.

3.5 Trade Direction Assignment

For each swap, the project assigns a binary direction:

- `direction = 0` : token0 flows into the pair

- `direction = 1` : token1 flows into the pair

When the input side is clear, the direction is inferred directly from the non-zero input field. Otherwise, decimal-normalized token inputs are compared so that different token units do not distort the direction assignment.

3.6 Price Reconstruction and Valuation Setup

To estimate profits in USD terms, the project builds a historical ETH price index from Uniswap V2 reserves. It first uses `WETH_USDC` reserve data. If this pair is unavailable, it falls back to `WETH_USDT` and then `WETH_DAI` .

Stablecoins such as `USDC` , `USDT` , and `DAI` are treated directly as USD-denominated assets. `WETH` is converted using the reconstructed ETH price series. `WBTC` is valued through a dynamic BTC/ETH ratio derived from the `WETH_WBTC` pool reserves, which is more appropriate for historical analysis than using a fixed approximation.

3.7 Output Summary

Before detector execution, the crawler produces the following event-level records:

- **Sync**: 1,349,490
- **Swap**: 1,315,097
- **Mint**: 13,381
- **Burn**: 12,505
- **Total**: **2,690,473**

The subsequent detector output contains:

- **845** sandwich events
- **758** displacement events
- **3,593** arbitrage / back-running events
- **18** suppression events

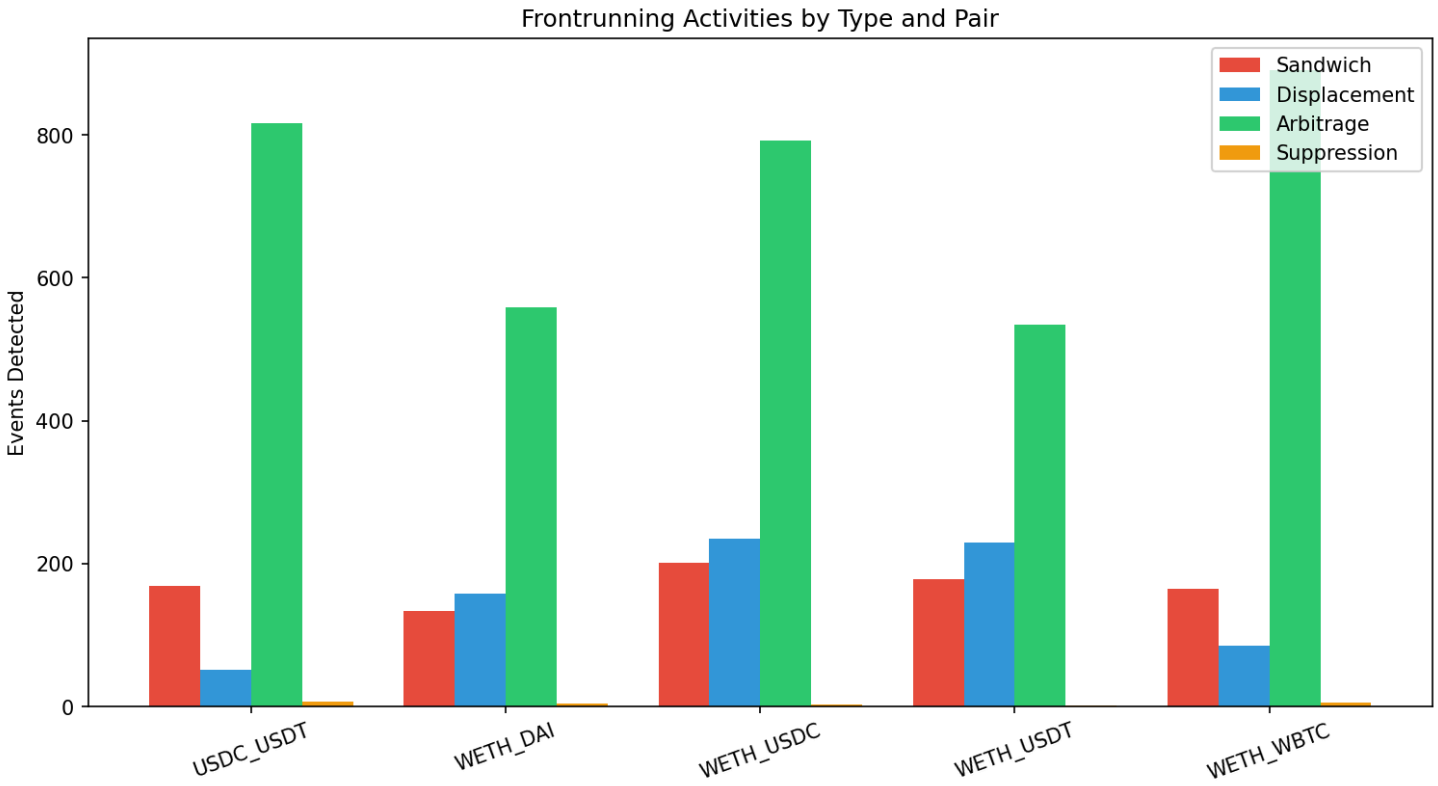


Figure 1. Detected frontrunning activities by type and trading pair.

These detection results form the empirical basis for the later discussion of profitability, gas-price behavior, and cross-pair differences.

4. Detection Method

4.1 Sandwich Attacks

4.1.1 Intuition

A sandwich attack occurs when one entity trades before a victim transaction and then trades again after it in the same block. The first trade opens a position, the victim transaction moves the AMM price, and the second trade closes the position after that price movement. The attacker is therefore attempting to profit from the price impact created by the victim.

4.1.2 Implementation Logic

The detector first identifies candidate blocks that contain at least three swaps and at least one repeated entity. Within each such block, swaps are grouped by entity, and the code searches for two swaps from the same entity with opposite directions.

A sandwich candidate is recorded when all of the following conditions are satisfied:

1. The same entity appears at least twice in the same block.
2. The two swaps have opposite directions.
3. The two swaps are different transactions.
4. At least one swap from a different entity lies strictly between them in execution order.
5. The intermediate swap has the same direction as the attacker's first trade.

These conditions are designed to recover the standard front-run → victim → back-run structure directly from ordered swap data.

4.1.3 Profit Calculation Logic

For each detected sandwich, the project computes the attacker's net token gain across the front-run and back-run legs. Let the attacker's net token balances after both legs be `net_token0` and `net_token1`. These balances are then converted into USD using token-specific valuation rules.

The gross profit is:

$$\mathrm{ProfitUSD} = \mathrm{USD}(n_0) + \mathrm{USD}(n_1)$$

Gas cost is estimated using a fixed gas model of 150,000 gas per swap:

$$\mathrm{GasCostUSD} = \frac{(g_{\mathrm{front}} + g_{\mathrm{back}}) \times 150000}{10^{18}} \times P_{\mathrm{ETH}}(B)$$

The final estimated net profit is:

$$\mathrm{NetProfitUSD} = \mathrm{ProfitUSD} - \mathrm{GasCostUSD}$$

Among the four detection categories, this is the most direct profit model because it evaluates the same-block round-trip position created by the attacker.

4.1.4 Findings

A total of **845** sandwich attacks were detected. Among them, **179** have positive estimated net profit, so profitable cases account for about **21.2%** of all detected sandwiches.

The aggregate sandwich profitability is:

- Profitable: **179** / 845 (21.2%)
- Gross profit: **$668,920.24**
- Net profit: **$194,984.86**
- Avg / Median net: $230.75 / -23.27$

Sandwich profits are clearly uneven rather than broadly distributed. Many detected cases become small or negative once gas cost is included, while a smaller set of profitable events contributes a large share of the total gains.

Pair-level sandwich counts are as follows:

Pair	Sandwiches	% Blocks	% Volume
WETH_USDC	201	0.08%	0.19%
WETH_USDT	178	0.08%	0.19%
WETH_DAI	133	0.05%	0.14%
WETH_WBTC	164	0.09%	0.25%
USDC_USDT	169	0.07%	0.21%

4.1.5 Figure

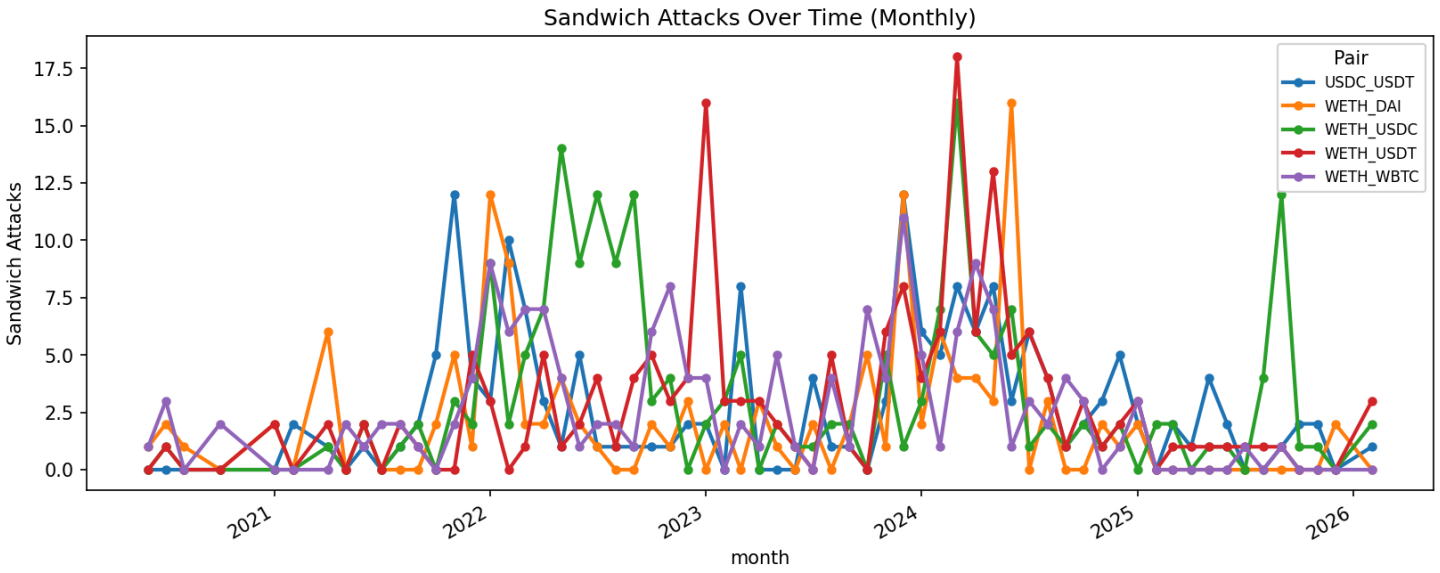


Figure 2. Monthly number of detected sandwich attacks across the analyzed pairs.

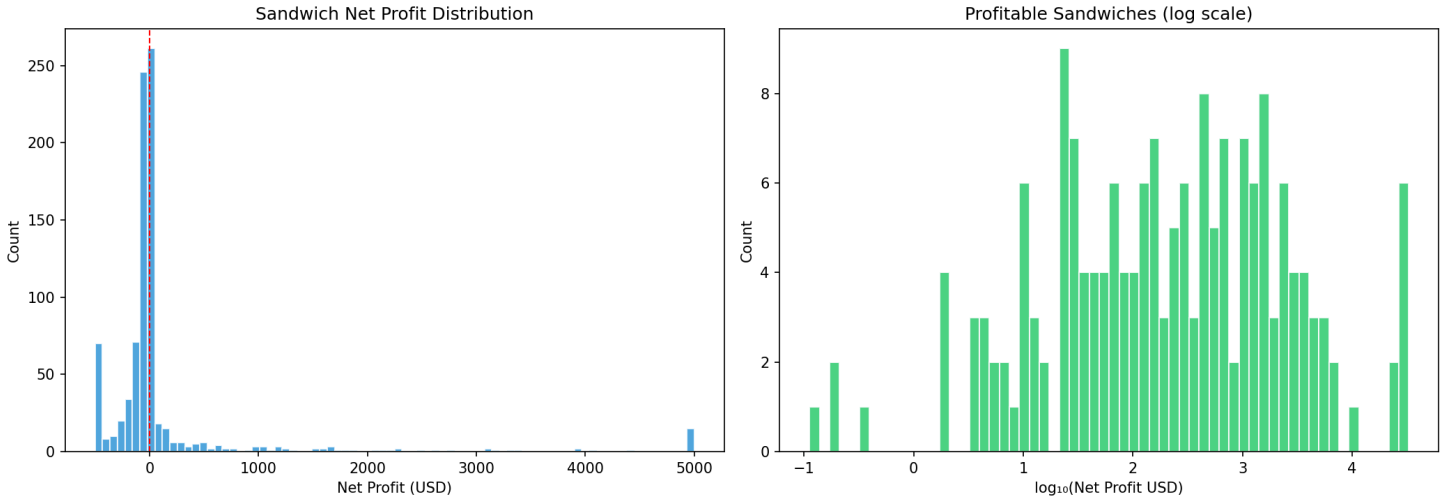


Figure 3. Distribution of estimated sandwich net profit, including the skewness of profitable cases.

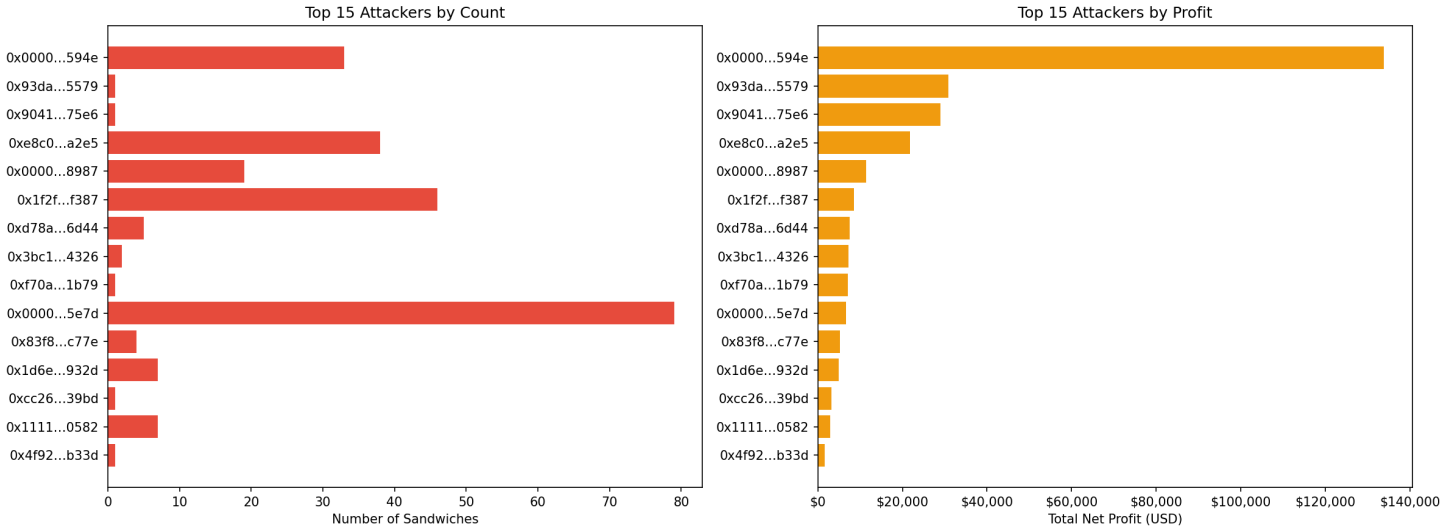


Figure 4. Top sandwich attackers by event count and cumulative estimated net profit.

4.2 Displacement Frontrunning

4.2.1 Intuition

Displacement frontrunning describes a case in which one trader gains execution priority over another trader attempting a similar trade. The two transactions move in the same direction, but the earlier one pays more gas and is included first, potentially leaving the later trader with a worse execution result.

4.2.2 Implementation Logic

The detector scans pairs of swaps inside the same block and flags a displacement event when all of the following conditions hold:

1. The two swaps belong to different entities.

2. The two swaps have the same direction.
3. The first swap executes earlier in the block (`tx_index` strictly smaller).
4. The transactions are within five transaction positions of each other.
5. The gas-price ratio satisfies:

$$\frac{g_f}{g_v} \geq 1.5$$

Here, the first transaction is treated as the potential frontrunner and the later transaction as the potential victim.

This is still a same-block heuristic rather than direct proof of frontrunning. Because mempool data are unavailable, the method cannot show whether the later trader originally expected to execute first. What it can show is a repeated pattern of close-proximity, same-direction trading accompanied by a meaningful gas-price advantage.

4.2.3 Value / Profit Interpretation

Compared with sandwich attacks, displacement is harder to evaluate because the outcome depends on a counterfactual question: how much worse was the later trade because it lost priority? For that reason, the report does not treat displacement profit as directly realized profit in the same sense as a completed sandwich round trip.

However, the code does estimate the economic effect of this ordering advantage. It reports:

- the USD value of the frontrunner's swap,
- the USD value of the victim's swap,
- the frontrunner's estimated gas cost,
- the victim's estimated loss under the counterfactual benchmark, and
- an estimated profit signal derived from that loss.

The interpretation should therefore remain cautious: displacement is best treated as an ordering-advantage pattern with an estimated economic effect, not as a clean realized arbitrage payoff.

4.2.4 Findings

A total of **758** displacement events were detected.

The gas-ratio statistics show strong skewness:

- Total displacement events: **758**
- Median gas ratio (frontrunner / victim): **3.2×**
- Pairs: WETH_USDC, WETH_USDT, WETH_DAI, WETH_WBTC, USDC_USDT

The gap between ordinary and extreme cases appears substantial. A few very large gas ratios pull the mean upward, while the median gives a more stable picture of typical same-block priority bidding.

Top 5 Displacement Frontrunners are:

Entity	Events
`0xfbd4cdb4…794c37`	82
`0x66a9893c…dba8af`	79
`0x3328f7f4…309c49`	58
`0x3fc91a3a…2b7fad`	21
`0x80a64c6d…cd5d9e`	20

4.2.5 Figure

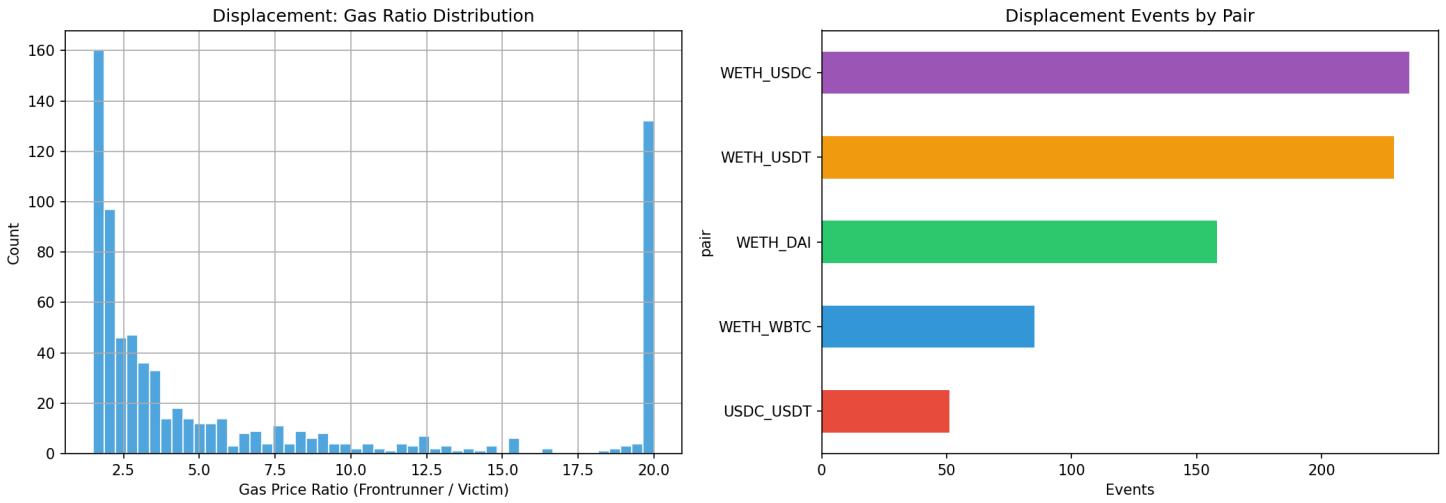


Figure 5. Distribution of displacement gas ratios and pair-level event counts.

4.3 Arbitrage / Back-running

4.3.1 Intuition

Arbitrage / back-running occurs when a trader reacts immediately after a large trade that has already changed the pool price. The trigger trade creates temporary price impact, and the back-runner exploits the resulting imbalance with an opposite-direction swap.

4.3.2 Implementation Logic

The detector first computes a trade-size measure for each swap as the larger of the two decimal-normalized token inputs. A swap is treated as a trigger trade if its size is above the pair-specific 90th percentile:

$$S(t) \geq Q_{0.90}(S)$$

For each trigger trade, the detector searches for a later swap in the same block such that:

1. it belongs to a different entity,
2. it has the opposite direction,
3. it occurs within the next three transaction positions.

When these conditions are satisfied, the follower is recorded as an arbitrage / back-running event.

This rule is meant to capture immediate reaction trades rather than broader multi-step arbitrage paths. In practice, it highlights traders who appear to respond quickly to a large price-moving transaction inside the same block.

4.3.3 Profit Calculation Logic

For each detected back-runner, the project computes the net token output of the reaction trade and converts it into USD. To avoid circular pricing, the system uses the ETH price from the **previous block** as the fair-market reference, since the trigger trade in the current block has already changed the pool reserves and distorted the same-block price.

If the back-runner receives more of a token than it sends, the difference contributes to estimated profit. The resulting gross value is then reduced by estimated gas cost:

$$\mathrm{NetProfitUSD} = \mathrm{ProfitUSD} - \mathrm{GasCostUSD}$$

where:

$$\mathrm{GasCostUSD} = \frac{g_{\mathrm{back}} \times 150000}{10^{18}} \times P_{\mathrm{ETH}}^{\mathrm{pre}}(B)$$

Here $P_{\mathrm{ETH}}^{\mathrm{pre}}(B)$ denotes the ETH price from the block immediately before block B, which serves as an undistorted market reference.

This provides a workable profit proxy for comparing back-running opportunities across the analyzed pairs.

4.3.4 Findings

A total of **3,593** arbitrage / back-running events were detected, making this the most frequent suspicious pattern in the dataset.

Estimated profitability is substantial:

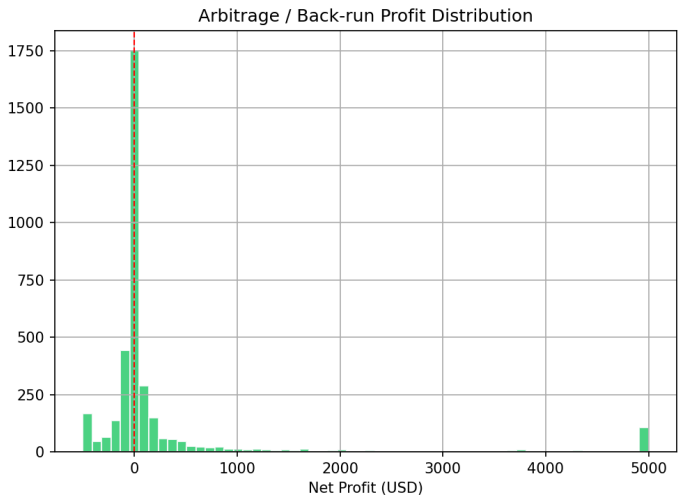
- Total back-run events: **3,593**
- Net profit: **$9,895,543.96**
- Avg net profit: **$2,754.12**

Top 5 Back-runners are:

Entity	Events
`0xa69babef…56e78c`	210
`0xa57bd001…fdd6cf`	179
`0x6b75d8af…009a80`	153
`0x51c72848…502a7f`	141
`0x860bd2db…d78f66`	120

Back-running dominates the dataset both in frequency and in aggregate estimated profitability. That pattern is consistent with AMM market structure: large swaps create short-lived local imbalances, and traders able to react within the same block can often exploit them quickly.

4.3.5 Figure



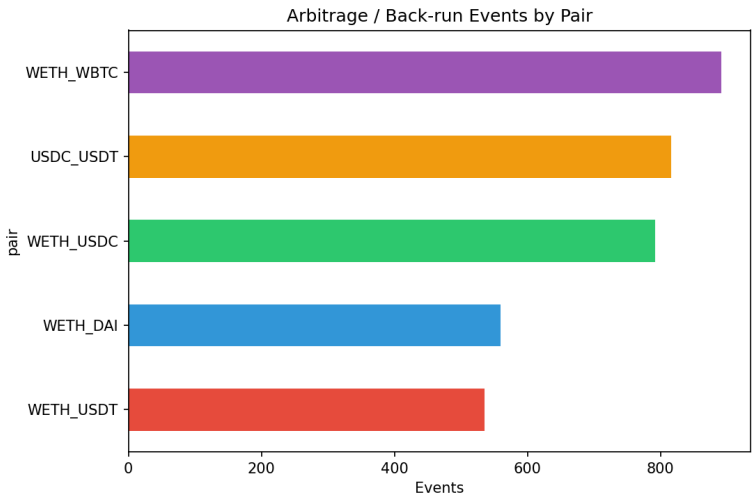


Figure 6. Estimated net-profit distribution and pair-level counts for detected arbitrage / back-running events.

4.4 Suppression

4.4.1 Intuition

Suppression refers to block-level behavior in which one entity submits multiple high-gas swap transactions within the same block, potentially crowding out other market participants or dominating block-level execution.

4.4.2 Implementation Logic

The detector first computes the median gas price for each block. It then aggregates swaps by `(block_number, entity)` and flags a suppression candidate when:

1. one entity submits at least three swaps in the same block, and
2. the entity's median gas price is at least three times the block median.

Formally, the gas-premium threshold is:

$$\mathrm{GasPremium} = \frac{\mathrm{EntityMedianGas}}{\mathrm{BlockMedianGas}} \geq 3.0$$

This detector is intentionally heuristic. It is designed to identify unusually aggressive same-block gas bidding, not to prove direct victim loss in every case.

4.4.3 Value / Profit Interpretation

Suppression does not have a directly recoverable realized-profit formula from swap logs alone. Its economic role is therefore more indirect than sandwich or back-running. A suppressor may be protecting another strategy, overwhelming nearby competition, or exploiting temporary block-level ordering dominance.

For that reason, the report interprets suppression mainly through **gas premium**, **swap concentration**, and **block dominance**, rather than exact realized net profit.

4.4.4 Findings

Only **18** suppression events were detected, making suppression the rarest pattern in the dataset.

However, the gas-premium signal is extremely strong:

- Total suppression events: **18**
- Median gas premium: **20.5×**

The distribution is heavily right-skewed, with a small number of extreme cases pulling the mean far above the median. This suggests that suppression is uncommon, but the observed cases are unusually aggressive in terms of gas-price competition.

Top 5 Suppressors are:

Entity	Events
`0x6b75d8af…009a80`	10
`0x1f2f10d1…6df387`	3
`0x00000000…120e49`	1
`0x3328f7f4…309c49`	1
`0x7c63795c…bcfde8`	1

4.4.5 Figure

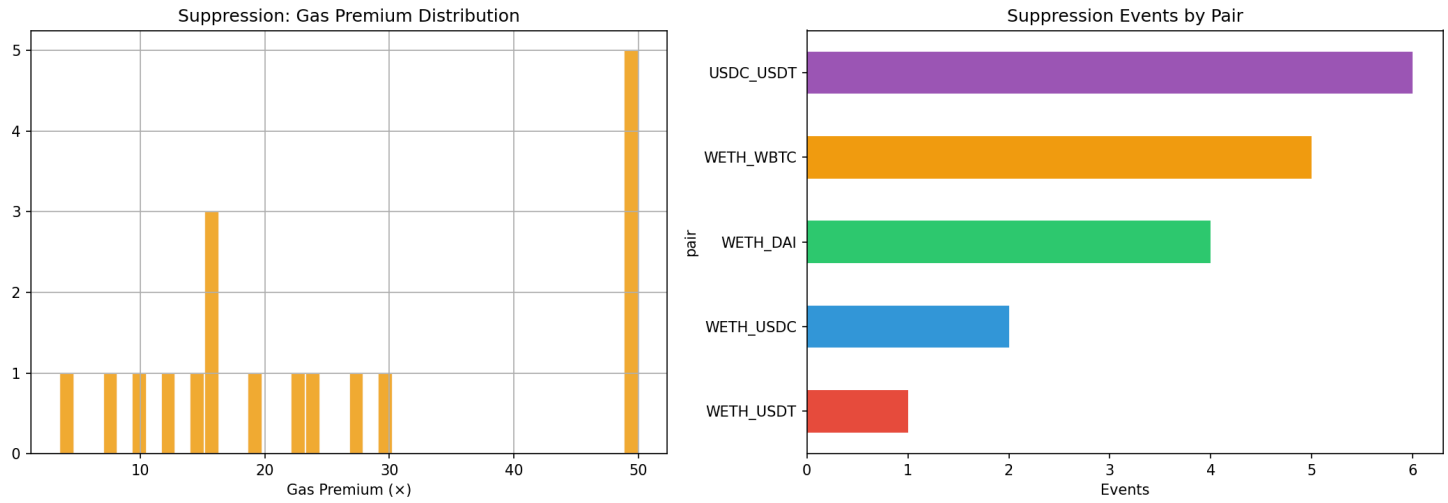


Figure 7. Gas-premium distribution and pair-level counts for detected suppression events.

5. Gas Price Analysis

Gas price is one of the most important on-chain proxies for transaction priority. Because MEV strategies depend heavily on execution order, gas-price behavior provides supporting evidence for strategic bidding and competition within blocks.

The gas analysis in this project compares gas-price patterns between suspicious blocks and ordinary blocks, especially for sandwich-related activity. The pair-level comparison shows that suspicious blocks often have noticeably higher median gas prices than normal blocks. This premium is modest in some pairs but extremely large in others, indicating that the intensity of gas competition differs across markets.

From the generated gas chart, the main pattern is clear:

- `WETH_USDC` and `WETH_USDT` show moderate gas premiums in sandwich-related blocks.
- `WETH_DAI`, `WETH_WBTC`, and `USDC_USDT` show much larger gas premiums.
- This supports the view that gas-price competition is an important supporting signal for MEV-related activity, particularly sandwich attacks and suppression-like behavior.

The gas analysis strengthens the broader interpretation of the detected events. Suspicious ordering patterns appear not only in transaction sequence, but often also in the gas prices paid to secure priority.

Figure

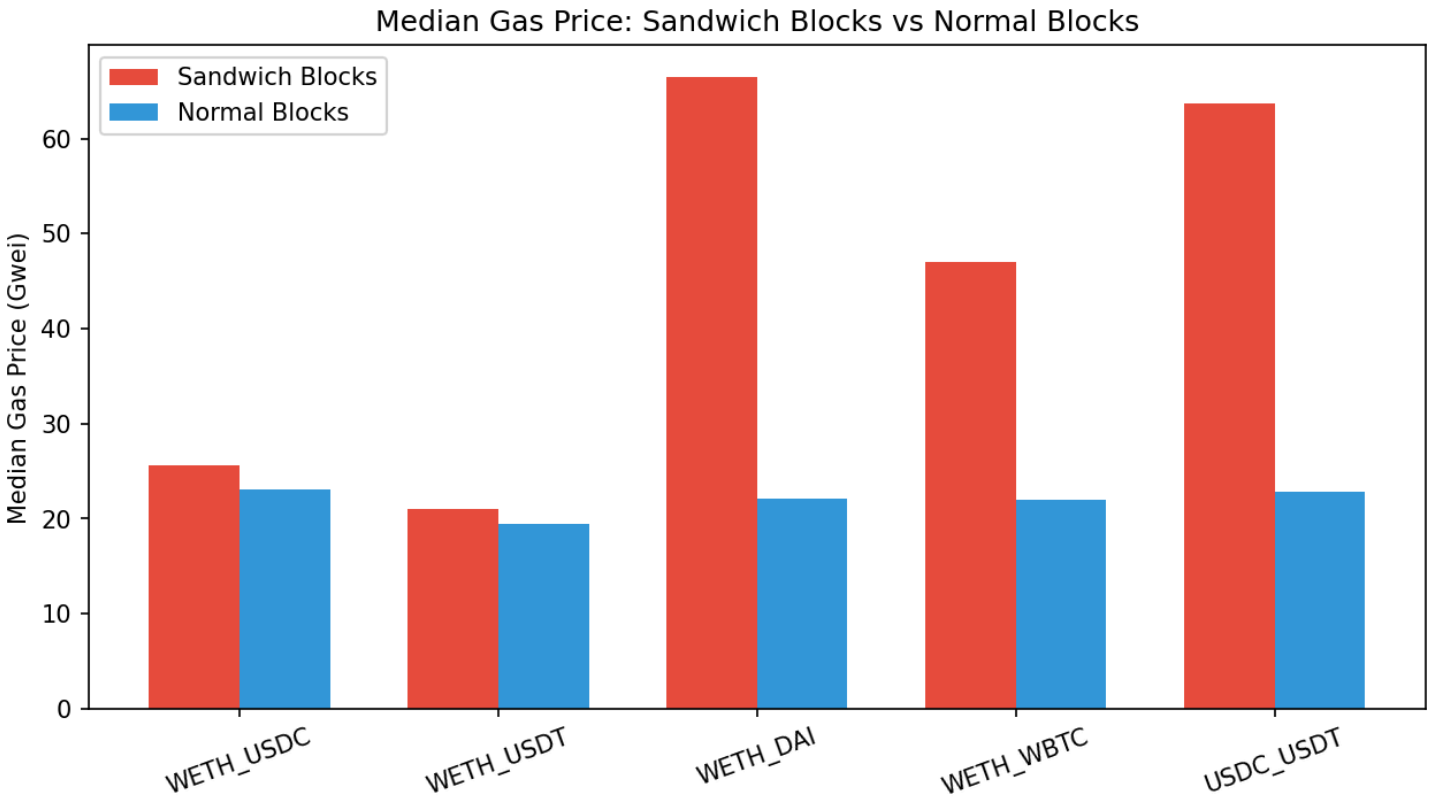


Figure 8. Median gas price in sandwich-related blocks versus normal blocks across the analyzed pairs.

6. Limitations

This study has several important limitations.

1. On-Chain Visibility Only

The analysis uses on-chain event data only. Failed, dropped, or replaced mempool transactions are not observable. As a result, the study cannot directly reconstruct the full competition process that occurred before final block inclusion.

2. Heuristic Entity Identification

Trader identity is approximated using a router-based heuristic. When a transaction is sent through a known router, the `to` address is treated as the actual entity; otherwise, the `sender` is used. This improves practical grouping, but it may still merge unrelated transactions or split activity that belongs to the same trader.

3. Simplified Gas Model

Gas cost is estimated using a fixed assumption of **150,000 gas per swap**. Actual gas usage may vary across transactions, so reported net profit values should be interpreted as estimates rather than exact realized outcomes.

4. Price Approximation

USD valuation relies on reserve-derived ETH prices. WBTC is converted using a dynamic BTC/ETH ratio reconstructed from the `WETH_WBTC` pool reserves, which provides historically grounded pricing. For arbitrage profit estimation, the system uses the previous block's ETH price as a pre-impact reference to avoid circular pricing. Even with these adjustments, approximation error may still arise in highly volatile periods.

5. Threshold Sensitivity

Several detection rules depend on heuristic thresholds, including:

- displacement gas ratio threshold (`1.5x`),
- arbitrage trigger threshold (pair-specific `90th percentile`),
- suppression gas premium threshold (`3x`),
- maximum same-block position gaps for displacement and arbitrage.

Changing these thresholds would change the number of detected events and may affect the balance between false positives and false negatives.

6. Interpretation Caution

The report identifies patterns that are strongly consistent with known MEV strategies, but it does not prove malicious intent in every case. Some flagged events may reflect ordinary reactive trading or legitimate high-priority execution rather than explicit predatory behavior.

7. Conclusion

This project builds a complete workflow for detecting and analyzing MEV-related frontrunning behavior on Uniswap V2 using historical Ethereum mainnet event data. By combining structured swap ordering, heuristic entity identification, and post-analysis of profits and gas prices, the system turns raw event logs into interpretable evidence of suspicious transaction-ordering behavior.

The four MEV patterns differ clearly in their empirical profile. Sandwich attacks are less frequent than back-running, and their profitability is concentrated in a relatively small share of events. Displacement frontrunning is better interpreted as an ordering-advantage pattern supported by gas asymmetry and counterfactual loss estimates than as a clean realized arbitrage payoff. Arbitrage / back-running is the most frequent pattern and contributes the largest aggregate estimated profit. Suppression is rare, but when it appears, it is associated with unusually strong gas-price premiums.

Overall, ordered swap data and gas-price behavior provide a useful basis for studying MEV on AMM-based decentralized exchanges. The approach is constrained by the absence of mempool visibility and by several heuristic assumptions, but it still offers a practical and reproducible framework for investigating transaction-ordering behavior on Uniswap V2.

8. References

[1] Frontrunner jones and the raiders of the dark forest: An empirical study of frontrunning on the ethereum blockchain. Usenix security 2021.

[2] Quantifying Blockchain Extractable Value: How dark is the forest? SP 2022.